

1. Error Analysis in Maximum Likelihood Estimation (MLE)

a) Identify the conceptual error in the likelihood formulation.

The student incorrectly defines the likelihood as the **sum of probabilities** instead of the **product of probabilities**.

Incorrect:

$$L(\theta) = \sum_{i=1}^n P(x_i | \theta)$$

Correct:

$$L(\theta) = \prod_{i=1}^n P(x_i | \theta)$$

The likelihood of independent observations is always the **product** of their individual probabilities.

b) Explain how this mistake affects parameter estimation.

- The sum does not represent the joint probability of the dataset.
- The estimated parameters no longer maximize the true likelihood.
- The model learns incorrect parameter values.
- Prediction accuracy decreases even with large datasets.

c) Suggest the correct mathematical formulation and reasoning.

For independent samples:

$$L(\theta) = \prod_{i=1}^n P(x_i | \theta)$$

The Maximum Likelihood Estimator (MLE) finds the parameter that maximizes this likelihood.

The product correctly represents the probability of observing the entire dataset.

d) How would using log-likelihood fix computational issues?

Take the logarithm:

$$\log L(\theta) = \sum_{i=1}^n \log P(x_i | \theta)$$

Advantages:

- Converts multiplication into addition.
- Prevents numerical underflow.
- Easier to differentiate.
- Simplifies optimization using gradient-based methods.

2. Error Diagnosis in Perceptron Learning

a) List possible implementation errors in the learning algorithm.

Possible errors include:

- Incorrect weight update formula.
- Missing bias term.
- Wrong learning rate.
- Incorrect prediction function.
- Wrong class labels (should be +1 and -1).
- Errors in stopping condition.

b) Analyze how incorrect learning rate or weight updates affect convergence.

- **Very large learning rate:** Overshoots the optimal solution and causes oscillation.
- **Very small learning rate:** Makes learning extremely slow.
- **Incorrect weight update:** Prevents the Perceptron from finding the correct decision boundary.

Correct update rule:

$$w = w + \eta(y - \hat{y})x$$

where:

- η = learning rate
- y = actual label
- $\hat{y}$ = predicted label

c) Identify dataset-related issues that may cause this problem.

Possible issues:

- Data is not actually linearly separable.
- Incorrect labels.
- Presence of noisy samples.
- Unscaled feature values.
- Duplicate or inconsistent training examples.

d) Propose modifications to ensure convergence.

- Normalize features.
- Choose an appropriate learning rate.
- Verify correct weight update implementation.
- Include the bias term.
- Remove noisy or mislabeled samples.
- Confirm that the dataset is linearly separable.

3. Backpropagation Gradient Error Analysis

a) Identify possible mathematical or coding errors in gradient computation.

Possible errors:

- Wrong derivative of activation function.
- Incorrect chain rule implementation.
- Incorrect sign in gradient update.
- Learning rate too large.

- Wrong matrix dimensions.
- Incorrect loss function derivative.

b) Explain the role of activation functions in gradient flow.

Activation functions introduce nonlinearity.

Examples:

- Sigmoid
- Tanh
- ReLU

During backpropagation, gradients are multiplied by the derivative of the activation function.

If derivatives become very small, gradients vanish.

If gradients become very large, they explode.

c) Analyze how vanishing or exploding gradients affect training.

Vanishing Gradients

- Gradients become close to zero.
- Early layers stop learning.
- Training becomes very slow.

Exploding Gradients

- Gradients become extremely large.
- Weight updates become unstable.
- Loss increases rapidly.
- Training may diverge.

d) Suggest techniques to fix the issue.

- Xavier or He weight initialization.
- Batch Normalization.
- Gradient Clipping.

- ReLU activation.
- Reduce learning rate.
- Use Adam optimizer.

These methods stabilize gradients and improve convergence.

4. Gradient Descent vs SGD Error Investigation

a) Explain why Batch Gradient Descent is slow in large datasets.

Batch Gradient Descent computes gradients using the **entire dataset** before updating weights.

Therefore:

- High computational cost.
- Large memory usage.
- Slow parameter updates.

b) Analyze the cause of fluctuations in SGD.

SGD updates weights after **each training example**.

Advantages:

- Faster updates.
- Escapes shallow local minima.

Disadvantages:

- High variance in gradients.
- Noisy updates.
- Loss fluctuates during training.

c) Compare the error behavior of Mini-Batch Gradient Descent.

Mini-Batch Gradient Descent uses small batches (e.g., 32 or 64 samples).

Advantages:

- Faster than Batch GD.

- More stable than SGD.
- Lower gradient variance.
- Better hardware utilization.

Hence, it provides a balance between speed and stability.

d) Recommend the best approach and justify with error trade-offs.

Mini-Batch Gradient Descent is generally the best choice because it:

- Converges faster than Batch GD.
- Produces smoother updates than SGD.
- Reduces computation time.
- Improves convergence stability.
- Achieves better generalization.

5. Curse of Dimensionality Error Analysis

a) Explain how high dimensionality increases model error.

As the number of features increases:

- Data becomes sparse.
- More parameters must be learned.
- The model memorizes training data.
- Prediction error on new data increases.

This leads to overfitting.

b) Identify signs of overfitting in this scenario.

Signs include:

- Very high training accuracy.
- Low test accuracy.
- Large gap between training and testing performance.
- Poor generalization to unseen data.

c) Analyze why distance-based learning fails in high dimensions.

Algorithms such as KNN rely on distance calculations.

In high-dimensional spaces:

- Distances between points become very similar.
- Nearest neighbors become less meaningful.
- Similarity measures lose effectiveness.
- Classification accuracy decreases.

This is known as the **Curse of Dimensionality**.

d) Suggest dimensionality reduction or regularization techniques to improve performance.

Dimensionality Reduction

- Principal Component Analysis (PCA)
- Linear Discriminant Analysis (LDA)
- Autoencoders

These reduce the number of features while preserving important information.

Regularization

- L1 Regularization (LASSO)
- L2 Regularization (Ridge)
- Dropout (for neural networks)

These methods reduce model complexity, prevent overfitting, and improve generalization.